

```

1  #include <iostream>
2  #include <vector>
3  #include <queue>
4  using namespace std;
5
6  struct TreeNode {
7      int val;
8      TreeNode* left;
9      TreeNode* right;
10
11     TreeNode(int x) : val(x), left(NULL), right(NULL) {}
12 };
13
14  TreeNode* buildTree(const vector<int>& nodes) {
15     if (nodes.empty() || nodes[0] == -1) return NULL;
16
17     TreeNode* root = new TreeNode(nodes[0]);
18     queue<TreeNode*> q;
19     q.push(root);
20
21     int i = 1;
22     while (!q.empty() && i < nodes.size()) {
23         TreeNode* current = q.front();
24         q.pop();
25
26

```

```

26
27 = if (nodes[i] != -1) {
28     current->left = new TreeNode(nodes[i]);
29     q.push(current->left);
30 }
31 i++;
32
33 = if (i < nodes.size() && nodes[i] != -1) {
34     current->right = new TreeNode(nodes[i]);
35     q.push(current->right);
36 }
37 i++;
38 }
39
40 return root;
41 }
42
43 = void reverseInorder(TreeNode* root, int k, int& count, int& result) {
44     if (!root || count >= k) return;
45
46     reverseInorder(root->right, k, count, result);
47
48     count++;
49 = if (count == k) {
50     result = root->val;
51     return;
52 }
53

```

```
53  
54     reverseInorder(root->left, k, count, result);  
55 }  
56  
57 int findKthLargest(TreeNode* root, int k) {  
58     int count = 0, result = -1;  
59     reverseInorder(root, k, count, result);  
60     return result;  
61 }  
62  
63 int main() {  
64     int T;  
65     cin >> T;  
66  
67     while (T--) {  
68         int n, val, k;  
69         cin >> n;  
70  
71         vector<int> nodes;  
72         for (int i = 0; i < n; ++i) {  
73             cin >> val;  
74             nodes.push_back(val);  
75         }  
76  
77         cin >> k;  
78
```

```
57 int findKthLargest(TreeNode* root, int k) {
58     int count = 0, result = -1;
59     reverseInorder(root, k, count, result);
60     return result;
61 }
62
63 int main() {
64     int T;
65     cin >> T;
66
67     while (T--) {
68         int n, val, k;
69         cin >> n;
70
71         vector<int> nodes;
72         for (int i = 0; i < n; ++i) {
73             cin >> val;
74             nodes.push_back(val);
75         }
76
77         cin >> k;
78
79         TreeNode* root = buildTree(nodes);
80         cout << findKthLargest(root, k) << endl;
81     }
82
83     return 0;
84 }
```

1

5

2 4 1 6 8

3

8